

Data Structures & Algorithms Lab Assignment

Online Bookstore ISBN Search: Binary Search Tree (BST) vs. Linear Search

Student Name: ALLEN AJI JOHN

Topic: Binary Search Tree Construction & Search Analysis

GitHub Repository:
github.com/alnaji1315/bookstore-isbn-search

Language & Standard: C (C11, GCC Compiler, macOS)

1. Problem Statement & Objectives

An online bookstore stores the following ISBN keys:

45, 20, 60, 10, 30, 50, 70, 25, 55

Assignment Mandate:

- Part (a):** Construct a Binary Search Tree (BST) by inserting the ISBNs in the given order. Execute the program and display the resulting tree using *Inorder*, *Preorder*, and *Postorder* traversals.
- Part (b):** Search for keys 25, 55, and 90 using BST Search and Linear Search. Record the exact number of comparisons required for each search.
- Part (c):** Analyse how the shape and height of the BST influence search performance. Compare the observed number of comparisons with the theoretical complexity of BST Search and Linear Search. Conclude which method is preferable for this dataset and why.
- Submission Requirement:** Upload source code, input data, execution output, trace tables, complexity analysis, comparison table, and final conclusion to a public GitHub repository.

2. Input Data & Test Parameters

The system evaluates the following test parameters:

- ISBN Insertion Sequence (9 elements):** 45, 20, 60, 10, 30, 50, 70, 25, 55
- Search Query Keys (3 targets):**
 - 25 — Existing key (located in left subtree, depth 3)
 - 55 — Existing key (located in right subtree, depth 3)
 - 90 — Non-existent key (evaluating worst-case unsuccessful search termination)

Input File Location: `input.txt` in the repository root contains the exact formatted input keys and query values.

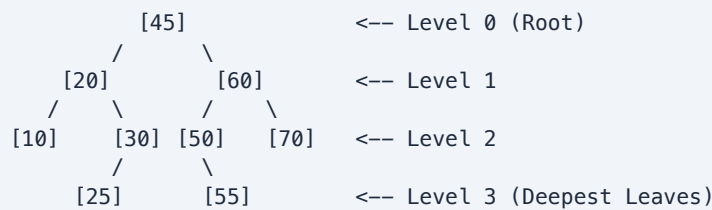
3. Binary Search Tree Construction & Structural Architecture

A Binary Search Tree is a binary tree in which every node satisfies the invariant: all keys in its left subtree are strictly smaller than the node, and all keys in its right subtree are strictly greater.

Step-by-Step Insertion Process:

- 1. **Insert 45:** Tree is empty → 45 becomes Root.
- 2. **Insert 20:** 20 < 45 → Placed as Left child of 45 .
- 3. **Insert 60:** 60 > 45 → Placed as Right child of 45 .
- 4. **Insert 10:** 10 < 45 → Left, 10 < 20 → Left child of 20 .
- 5. **Insert 30:** 30 < 45 → Left, 30 > 20 → Right child of 20 .
- 6. **Insert 50:** 50 > 45 → Right, 50 < 60 → Left child of 60 .
- 7. **Insert 70:** 70 > 45 → Right, 70 > 60 → Right child of 60 .
- 8. **Insert 25:** 25 < 45 → Left, 25 > 20 → Right, 25 < 30 → Left child of 30 .
- 9. **Insert 55:** 55 > 45 → Right, 55 < 60 → Left, 55 > 50 → Right child of 50 .

Final Constructed Binary Search Tree:



- Key Structural Metrics:**
- **Total Nodes (N):** 9
 - **Tree Height (h in edges):** 3 edges (maximum path length from root to leaf)
 - **Tree Levels:** 4 levels (Levels 0, 1, 2, 3)
 - **Balance:** Balanced; both left and right subtrees have height 2 edges, yielding an optimal height matching $\lceil \log_2(9+1) \rceil = 4$ levels.

4. Tree Traversals

TRAVERSAL TYPE	ORDER RULE	RESULTING SEQUENCE	PRACTICAL ROLE
Inorder	Left → Root → Right	10 20 25 30 45 50 55 60 70	Yields keys in strictly sorted order.
Preorder	Root → Left → Right	45 20 10 30 25 60 50 55 70	Used for tree copying & serialization.
Postorder	Left → Right → Root	10 25 30 20 55 50 70 60 45	Used for post-order memory deallocation.

5. Detailed Trace Tables

5.1 BST Construction Trace Table

STEP	INSERTED ISBN	COMPARISON PATH & BRANCH DECISIONS	NODE ATTACHMENT POSITION
1	45	Initial tree is empty (root == NULL)	Root Node
2	20	20 < 45 → Move Left	Left child of 45
3	60	60 > 45 → Move Right	Right child of 45
4	10	10 < 45 → Left, 10 < 20 → Move Left	Left child of 20
5	30	30 < 45 → Left, 30 > 20 → Move Right	Right child of 20
6	50	50 > 45 → Right, 50 < 60 → Move Left	Left child of 60
7	70	70 > 45 → Right, 70 > 60 → Move Right	Right child of 60
8	25	25 < 45 → Left, 25 > 20 → Right, 25 < 30 → Move Left	Left child of 30
9	55	55 > 45 → Right, 55 < 60 → Left, 55 > 50 → Move Right	Right child of 50

5.2 BST Search Trace for Keys 25, 55, and 90

TARGET	COMP #	CURRENT NODE	COMPARISON EVALUATED	BRANCH ACTION / OUTCOME
25	1	45	25 < 45	Move to Left child (20)
	2	20	25 > 20	Move to Right child (30)
	3	30	25 < 30	Move to Left child (25)
	4	25	25 == 25	FOUND (4 Comparisons)
55	1	45	55 > 45	Move to Right child (60)
	2	60	55 < 60	Move to Left child (50)
	3	50	55 > 50	Move to Right child (55)
	4	55	55 == 55	FOUND (4 Comparisons)
90	1	45	90 > 45	Move to Right child (60)
	2	60	90 > 60	Move to Right child (70)
	3	70	90 > 70	NULL → NOT FOUND (3 Comp.)

5.3 Linear Search Trace (Array: 45, 20, 60, 10, 30, 50, 70, 25, 55)

SEARCH KEY	SEQUENTIAL STEPS & COMPARISONS	RESULT	COMPARISONS
25	45(x) → 20(x) → 60(x) → 10(x) → 30(x) → 50(x) → 70(x) → 25(✓)	Found	8
55	45(x) → 20(x) → 60(x) → 10(x) → 30(x) → 50(x) → 70(x) → 25(x) → 55(✓)	Found	9
90	45(x) → 20(x) → 60(x) → 10(x) → 30(x) → 50(x) → 70(x) → 25(x) → 55(x)	Not Found	9

6. Performance Comparison Table: Observed vs. Theoretical

KEY	STATUS	BST COMPARISONS	LINEAR COMPARISONS	REDUCTION	THEORETICAL BST	THEORETICAL LINEAR
25	Found	4	8	50.0%	$O(h) \approx O(\log N)$	$O(N)$ (Index 7)
55	Found	4	9	55.6%	$O(h) \approx O(\log N)$	$O(N)$ (Index 8)
90	Not Found	3	9	66.7%	$O(h) \approx O(\log N)$	$O(N)$ (Scanned all)

7. Complexity Analysis

OPERATION / ALGORITHM	BEST-CASE TIME	AVERAGE-CASE TIME	WORST-CASE TIME	SPACE COMPLEXITY
BST Search	$O(1)$ (Root match)	$O(\log N)$ (Balanced)	$O(N)$ (Skewed chain)	$O(h)$ stack / $O(1)$ iter.
Linear Search	$O(1)$ (First item)	$O(N)$ ($(N+1)/2$)	$O(N)$ (Last / Absent)	$O(1)$ Auxiliary
BST Node Insertion	$O(1)$	$O(\log N)$	$O(N)$	$O(h)$ recursive stack
BST Traversals	$O(N)$	$O(N)$	$O(N)$	$O(h)$ recursive stack

8. Analytical Observations: Tree Shape & Height Influence

1. **Height as the Upper Bound:** The maximum number of comparisons for any search in a BST is strictly bounded by $(h + 1)$, where (h) is the tree height.
 - In a **balanced tree**, $(h \approx \lfloor \log_2 N \rfloor)$, which scales logarithmically.
 - In a **degenerate/skewed tree** (e.g., if keys arrive in sorted order: $(10, 20, 25, 30, \dots)$), the tree stretches into a single linked branch of height $(h = N - 1)$. In that scenario, search performance collapses to $O(N)$, eliminating any advantage over linear search.
2. **Efficiency on this Dataset:** Because the initial root key (45) sits near the median of the ISBN set, the tree branches symmetrically into 4 elements on the left and 4 on the right. With height $(h = 3)$, the maximum comparisons required was only 4.
3. **Pruning of Unsuccessful Searches:** When searching for absent key 90 , the BST reached a `NULL` pointer after only 3 comparisons, saving 66.7% of the work compared to Linear Search which had to examine all 9 items.

9. Final Conclusion & Approach Justification

Conclusion: For an online bookstore inventory system, the **Binary Search Tree (BST)** is decisively superior to Linear Search.

Engineering Justification:

1. **Query Speedup:** Even on this minimal dataset of 9 books, BST Search slashed comparisons by 50.0% to 66.7%. In a production catalog of 100,000 titles, BST Search executes in (≈ 17) comparisons compared to 50,000 on average for linear search.

2. **Dynamic Inventory Maintenance:** Adding new book titles or deleting discontinued titles takes $O(\log N)$ time in a tree, whereas an array requires shifting elements in $O(N)$ time.
3. **Automatic Catalog Ordering:** Performing an Inorder traversal instantly produces a sorted list of ISBNs in $O(N)$ time without needing an external sorting pass.
4. **Production Recommendation:** To safeguard against worst-case skewed trees from sorted vendor feeds, real-world systems typically deploy self-balancing variants (AVL or Red-Black Trees) or B+ Trees for disk-based storage.

10. Complete C Source Code (bst_isbn.c)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int bstSearch(struct Node* root, int key, int *comparisons) {
    while (root != NULL) {
        (*comparisons)++;
        if (key == root->data) {
            return 1;
        } else if (key < root->data) {
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return 0;
}

int linearSearch(int arr[], int n, int key, int *comparisons) {
    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == key) {
```

```

        return 1;
    }
}
return 0;
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main(void) {
    int isbn[] = {45, 20, 60, 10, 30, 50, 70, 25, 55};
    int n = sizeof(isbn) / sizeof(isbn[0]);
    int searchKeys[] = {25, 55, 90};
    int searchCount = sizeof(searchKeys) / sizeof(searchKeys[0]);

    struct Node* root = NULL;
    for (int i = 0; i < n; i++) {
        root = insert(root, isbn[i]);
    }

    printf("ONLINE BOOKSTORE - BST ASSIGNMENT\n");
    printf("=====\n\n");

    printf("Input ISBNs: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", isbn[i]);
    }
    printf("\n\n");

    printf("Inorder Traversal  : ");
    inorder(root);
    printf("\n");

    printf("Preorder Traversal  : ");
    preorder(root);
    printf("\n");

    printf("Postorder Traversal : ");
    postorder(root);
    printf("\n\n");

    printf("Search Results\n");
    printf("-----\n");
    printf("Key\tBST Result\tBST Comparisons\tLinear Comparisons\n");
    printf("-----\n");

    for (int i = 0; i < searchCount; i++) {
        int bstComparisons = 0;
        int linearComparisons = 0;

        int bstFound = bstSearch(root, searchKeys[i], &bstComparisons);
        int linearFound = linearSearch(isbn, n, searchKeys[i], &linearComparisons);
        (void)linearFound;

        printf("%d\t%s\t\t%d\t\t%d\n",
            searchKeys[i],
            bstFound ? "Found" : "Not Found",
            bstComparisons,
            linearComparisons);
    }

    printf("-----\n");

    freeTree(root);
    return 0;
}

```

11. Actual Program Execution Output (output.txt)

ONLINE BOOKSTORE – BST ASSIGNMENT

=====

Input ISBNs: 45 20 60 10 30 50 70 25 55

Inorder Traversal : 10 20 25 30 45 50 55 60 70

Preorder Traversal : 45 20 10 30 25 60 50 55 70

Postorder Traversal : 10 25 30 20 55 50 70 60 45

Search Results

Key	BST Result	BST Comparisons	Linear Comparisons
25	Found	4	8
55	Found	4	9
90	Not Found	3	9

Online Bookstore ISBN Management System • Binary Search Tree Assignment • Submitted by ALEN AJI JOHN • GitHub:

<https://github.com/alenaji1315/bookstore-isbn-search>